

Accelerating Ray Tracing

By Garrett Doolittle and Ethan Hutton

The main goal for our project was to look into aspects of the code that could be optimized for increased performance. This means trying to speed up intersects, compress nodes, and cut out recursion when possible. The main methods by which we attempted to increase the efficiency of the system are listed below

Instructions for running our project:

Running our project is the same as running the base project.

If you want to adjust the packet size it can be done in the primitive file by adjusting the constant N.

Current State of the Art: The state of the art in our area is mostly centered around the use of GPUs. As seen in our accomplishments, parallelizing our tasks leads to a large increase in efficiency. GPUs are highly capable of running many tasks all at once and is pretty standard in many applications that try to accelerate their programs. There is also SIMD which we attempted to use in our Ray Tracer and will talk about more, but being able to utilize SIMD to its fullest extent is also key state of the art optimization that could be made in our area.

Stackless BVH: Recursion is costly, so finding a way to traverse the BVH without it will increase efficiency. To do this, you create a stack that checks the root node, if it hits and is not a leaf you push the left and right and then pop off the top of the stack to get the new pointer. The stack and traverse algorithm are in the StackBVHTree.

Storing Inverse directions in Rays: Intersecting with the AABBs is where you spend a lot of time. Inside the intersection you need the inverse direction to get the slabs in the current direction. Division can be costly, meaning that it's more efficient to put the inverse direction into the ray

where you don't have to divide each time you want to intersect some direction.

Faster Ray and Primitive Intersection: Intersecting AABBs and Triangles is most of the work that a ray tracer does. Optimizing these algorithms even slightly can result in substantial time improvement. The optimizations implemented in our ray tracer were [Möller-Trumbore](#) triangle intersection and [Graphic Gems](#) AABB intersection. These accounted for minor speed improvements.

OpenMP: By far the biggest decrease in time was including OpenMP into the makefile. OpenMp takes certain blocks of code and will allocate that code to different threads so that the tracer isn't all run on one thread. By putting '#pragma omp parallel for' before the for loop that runs through each pixel, you can parallelize each ray that's traced leading to a massive efficiency gain. The flags can be found in the CMake and the one pragma line that we run is in RayTracer.h

Roped BVH: The idea was taken from this paper here: <https://dl.acm.org/doi/pdf/10.1145/1980462.1980483>. While there is a bit more done in this paper with CUDA and Ray Packets, I wanted to try the general idea presented about each node having a rope. Instead of having a right child, each node will have a rope to its sibling node and start from there. This eliminates the need to recurse back to the parent to increase efficiency. This is not used in the final version since it has a similar run time to the stack, but I was curious if it would be any different. The objLoader and simpleLoader would need to be changed to get it to work (it might break with the compressed node now there), but as it is right now it is not used.

Compressed Nodes: Having smaller nodes, means that the processor can pull more nodes at a node. By allocating memory you can know where the right node is relative to the left node and eliminate the need to hold the right child in each node. This gave pretty inconsistent results, they were

sometimes faster, sometimes the same, and most of the time a little slower. Neither of us were really able to figure out why.

Sphere Intersections: Like AABB intersections, but with spheres. Since the spheres in theory are smaller in volume, there should technically be less hits than with a box leading to faster traversals. Ended up being really slow. If I had to guess as to why it would be because the intersects aren't as optimized compared to the AABB ones. A bit of a failed experiment, but it was worth a shot. This is not reflected in the final optimizations.

Ray Packets: We measured results against the base ray tracer given to us and our developing ray tracer. We took an average of the ray tracer we wanted to test against and ran it 3-5 times. We then compared it to the current branch that we had to see if there was an increase in performance.

SIMD Intersections: [SIMD](#) or Single Instruction Multiple Data takes advantage of Vector based calculation over the standard scalar calculations to do groups of the same calculation at once for multiple floats. Coding using SIMD is kinda similar to assembly because the operations that are possible are very limited. We attempted using SIMD to accelerate our ray tracer on the branch SIMD-attempt and successfully implemented all of the SIMD calculations but it ended up being much slower. We believe that the slowness is due to our limited SIMD coding experience and not knowing how to optimize it properly.

Here are some results with the bunny

All at 3000 x 3000 with -O3 flag and in Debug Mode

Base ray tracer: 2.681s, 2.840s, 2.699s

- Avg: 2.740s

Base + Compressed Nodes: 2.847s, 2.844s, 2.829s

- Avg: 2.838s

Prev + New Intersects: 2.742s, 2.741s, 2.755s

- Avg: 2.746s

Prev + Ray Packets: 2.550s, 2.429s, 2.489s

- Avg: 2.489s

Prev + StackBVH: 2.212s, 2.221s, 2.226s

- Avg: 2.220s

Prev + OMP + Other Optimization Flags: .693s, .687s, .708

- Avg: .696s

Overall a ~75% increase in performance and even without OMP it's ~20% increase in performance.

What Did We Learn:

- Parallelize tasks when possible
- It is more efficient to not use recursion
- Intersect is where a lot of time is spent, so optimizing it even a little is worth it

Next Steps:

- Convert the Ray Packets to SIMD properly
- Do actual GPU acceleration